

学生运动会系统

1. 系统概述

本系统是一个运动会管理系统，用于管理运动会的项目信息、学生信息以及学生参与项目的情况。系统提供了添加项目、添加学生、学生报名项目、查询学生参与项目情况、查询项目参与者以及展示所有项目信息及参与情况等功能。

2. 系统架构

- 主要包含以下几个类：
- **1.SportsEvent类：**
 - 作为基类，定义了所有体育相关类的基本接口。包含一个静态成员变量total，用于记录项目总数，以及纯虚函数display()和静态函数gets()。
- **2. Event类：**
 - 继承自SportsEvent类，用于表示具体的运动会项目。包含项目名称、类别和分值等属性，以及相应的构造函数、析构函数和显示函数。
- **3 Student类：**
 - 继承自SportsEvent类，用于表示参与运动会的学生。包含学号、姓名和参与的项目列表等属性，以及相应的构造函数、显示函数和报名项目的函数。
- **4. Management类：**
 - 作为系统的核心管理类，与Student类和Event类构成友元关系，负责管理所有学生和项目的信息，以及处理学生报名项目的业务逻辑。包含学生列表、项目列表和项目与学生的关联映射等属性，以及添加学生、添加项目、学生报名项目、查询学生参与情况、查询项目参与者和展示所有项目信息等功能函数。

3. 数据结构设计

- **1. Student类：**
 - id: 学生学号
 - name: 学生姓名
 - events: 学生参与的项目名称列表
- **2. Event类：**
 - name: 项目名称
 - category: 项目类别
 - score: 项目分值
- **3. Management类：**
 - students: 学生指针列表
 - events: 项目对象列表
 - eventJoin: 项目名称到参与学生指针列表的映射

4 功能模块设计

- **1. 添加项目功能:**
 - 输入项目名称、类别和分值
 - 创建项目对象并添加到项目列表中
 - 初始化项目的参与学生列表
- **2. 添加学生功能:**
 - 输入学生学号和姓名
 - 创建学生对象并添加到学生列表中
- **3. 学生报名项目功能:**
 - 输入学生学号和项目名称
 - 检查学生和项目是否存在
 - 若存在，将项目添加到学生的参与项目列表中，并将学生添加到项目的参与学生列表中
- **4. 查询学生参与情况功能:**
 - 输入学生学号
 - 查找学生并显示其基本信息
 - 显示学生参与的所有项目信息
- **5. 查询项目参与者功能:**
 - 输入项目名称
 - 查找项目并显示其参与者信息
- **6. 展示所有项目信息及参与情况功能:**
 - 遍历所有项目，显示每个项目的基本信息、参与人数和参与者列表

5. 系统流程

- **1. 初始化阶段:**
 - 创建管理对象
 - 输入项目数量并添加项目
 - 输入学生数量并添加学生
 - 输入记录数量并处理学生报名项目
- **2. 查询阶段:**
 - 输入要查询的学生数量并查询学生参与情况
 - 输入要查询的项目数量并查询项目参与者
- **3. 展示阶段:**
 - 显示总项目数
 - 展示所有项目信息及参与情况

6. 类设计详细说明

- **1. SportsEvent类:**
 - 基类，提供统一接口
 - 静态成员变量total用于记录项目总数
 - 纯虚函数display()要求派生类实现显示功能
 - 静态函数gets()用于获取项目总数
- **2. Event类:**
 - 继承自SportsEvent类
 - 构造函数初始化项目信息并增加项目总数
 - 拷贝构造函数复制项目信息并增加项目总数
 - 析构函数减少项目总数
 - 实现display()函数显示项目信息
- **3. Student类:**
 - 继承自SportsEvent类
 - 构造函数初始化学生信息
 - 拷贝构造函数复制学生信息
 - 实现display()函数显示学生信息
 - join()函数用于添加学生参与的项目
- **4. Management类:**
 - 管理所有学生和项目的信息
 - 析构函数释放学生对象的内存
 - addStudent()函数添加学生到学生列表
 - addEvent()函数添加项目到项目列表并初始化参与学生列表
 - joinEvent()函数处理学生报名项目的逻辑
 - queryStudent()函数查询学生参与情况
 - queryEvent()函数查询项目参与者
 - show()函数展示所有项目信息及参与情况

7. 数据流程

- 项目数据通过addEvent()函数添加到events列表中，并在eventJoin映射中初始化对应的学生列表
- 学生数据通过addStudent()函数添加到students列表中
- 学生报名项目时，通过joinEvent()函数更新学生的events列表和eventJoin映射中对应的学生列表
- 查询操作通过遍历students列表、events列表和eventJoin映射来获取相应信息
- 展示操作通过遍历events列表和eventJoin映射来显示所有项目信息及参与情况

8. 系统交互流程

- 用户输入项目信息，系统创建项目对象并存储
- 用户输入学生信息，系统创建学生对象并存储
- 用户输入学生报名项目信息，系统处理报名逻辑
- 用户查询学生参与情况，系统查找并显示相关信息
- 用户查询项目参与者，系统查找并显示相关信息
- 系统展示所有项目信息及参与情况

9. 异常处理

- 在学生报名项目时，若学生或项目不存在，系统会提示错误信息
- 在查询学生参与情况时，若学生不存在，系统会提示错误信息
- 在查询项目参与者时，若项目不存在，系统会提示错误信息

10. 总结

- 代码完成一下要求：
 - 设计包含一个以上的类
 - 类中包含构造函数、拷贝构造函数
 - 类中包含静态成员和静态成员函数
 - 类中包含友元
 - 继承和派生
 - 虚函数和虚基类
 - Main函数
- 系统提供了完善的查询和展示功能，具有良好的扩展性和可维护性。

11. 代码实现

```
1  #include<bits/stdc++.h>
2  using namespace std;
3
4  // 运动会项目基类（抽象类）
5  class SportsEvent
6  {
7      protected:
8          static int total;    // 静态成员：统计所有项目数量
9      public:
10         virtual void display() const = 0;    // 纯虚函数，强制派生类实现多态接口
11         static int gets(){return total;};
12 };
13
14 int SportsEvent::total=0;    // 静态成员初始化
15
16 // 具体项目类
17 class Event:public SportsEvent
18 {
19     string name;    // 项目名称
20     string category;    // 项目类别
```

```

21     int score; // 项目分值
22     friend class Management; // 友元声明：允许Management访问私有成员
23 public:
24     // 构造/拷贝构造时自动增加项目计数
25     Event(string n,string cat,int sc):name(n),category(cat),score(sc)
    {total++;}
26     Event(const Event&
    E):name(E.name),category(E.category),score(E.score){total++;}
27     ~Event(){total--;} // 析构时减少计数
28
29     void display()const override {cout<<"项目: "<<name<<" 类别: "
    <<category<<" 分值: "<<score<<endl;}
30 };
31
32 // 学生参与类
33 class Student:public SportsEvent
34 {
35     protected:
36         string id; // 学号
37         string name; // 姓名
38         vector<string> events; // 参与的项目名称列表
39     public:
40         Student(string sid,string n):id(sid),name(n){}
41         Student(const Student&
    Students):id(Students.id),name(Students.name){}
42
43         void display()const override {cout<<"学号: "<<id<<" 姓名: "
    <<name<<endl;}
44         void join(string eventname) {events.push_back(eventname);} // 记录
    学生参与的项目
45         friend class Management; // 友元声明
46     };
47
48 // 运动会管理系统
49 class Management {
50     vector<Student*> students; // 学生对象指针集合
51     vector<Event> events; // 项目对象集合
52     map<string, vector<Student*>> eventJoin; // 项目名称-参与者映射（高效查询）
53 public:
54     ~Management() {
55         for (auto s : students) delete s; // 析构时释放学生对象内存
56     }
57
58     // 添加学生到系统
59     void addStudent(Student* s) {
60         students.push_back(s);
61     }
62
63     // 创建新项目
64     void addEvent(string n, string c, int s) {
65         events.emplace_back(n, c, s); // 直接构造避免拷贝
66         eventJoin[n] = vector<Student*>(); // 初始化空参与列表
67     }
68
69     // 学生参加项目（双向关联）
70     void joinEvent(string studentId, string eventName) {

```

```

71 // Lambda表达式查找学生和项目
72 auto studentIt = find_if(students.begin(), students.end(),
73     [&](Student* s) { return s->id == studentId; });
74 auto eventIt = find_if(events.begin(), events.end(),
75     [&](const Event& e) { return e.name == eventName; });
76
77 if (studentIt != students.end() && eventIt != events.end()) {
78     (*studentIt)->join(eventName); // 更新学生参与列表
79     eventJoin[eventName].push_back(*studentIt); // 更新项目参与者
80     cout << "学生 " << (*studentIt)->name << " 成功参加项目 " <<
eventName << endl;
81 } else {
82     cout << "学生或项目不存在!" << endl;
83 }
84 }
85
86 // 查询学生参与情况
87 void queryStudent(string id) {
88     auto it = find_if(students.begin(), students.end(),
89         [&](Student* s) { return s->id == id; });
90
91     if (it != students.end()) {
92         (*it)->display();
93         cout << "参与项目: " << endl;
94         for (auto& eventName : (*it)->events) {
95             auto eventIt = find_if(events.begin(), events.end(),
96                 [&](const Event& e) { return e.name == eventName; });
97             if (eventIt != events.end())
98                 eventIt->display();
99             else
100                 cout << "-项目" << eventName << "信息缺失" << endl;
101         }
102     } else cout << "学生不存在";
103 }
104
105 // 查询项目参与情况
106 void queryEvent(string evename) {
107     if (eventJoin.find(evename) != eventJoin.end()) {
108         cout << "项目" << evename << "的参与者: " << endl;
109         for (auto student : eventJoin[evename])
110             student->display();
111         cout << "总计: " << eventJoin[evename].size() << "人" << endl;
112     } else cout << "项目不存在!" << endl;
113 }
114
115 // 显示所有项目及参与统计
116 void show() {
117     cout << endl << "所有项目信息及参与情况: " << endl;
118     for (auto& event : events) {
119         cout << "项目名称: " << event.name << endl;
120         cout << "参与人数: " << eventJoin[event.name].size() << endl;
121         if (!eventJoin[event.name].empty()) {
122             cout << "参与者: ";
123             for (auto student : eventJoin[event.name])
124                 cout << student->name << " ";
125             cout << endl;

```

```

126         }
127         cout << endl;
128     }
129 }
130 };
131 // 主程序逻辑：数据输入→关系建立→查询→统计
132 int main()
133 {
134     Management m;
135     int eventCount, studentCount, joinCount;
136     cout<<"请输入项目数量: ";
137     cin>>eventCount;
138     for(int i=0;i<eventCount;i++)
139     {
140         string name,cate;
141         int sc;
142         cout<<endl<<"项目"<<i+1<<"名称: "; cin>>name;
143         cout<<"项目"<<i+1<<"类别: "; cin>>cate;
144         cout<<"项目"<<i+1<<"分值: "; cin>>sc;
145         m.addEvent(name,cate,sc);
146     }
147
148     cout<<endl<<"请输入学生数量: ";
149     cin>>studentCount;
150     for(int i=0;i<studentCount;i++)
151     {
152         string id,name;
153         cout<<endl<<"学生"<<i+1<<"学号: "; cin>>id;
154         cout<<"学生"<<i+1<<"姓名: "; cin>>name;
155         m.addStudent(new Student(id,name));
156     }
157
158     cout<<endl<<"请输入记录数量: ";
159     cin>>joinCount;
160     for(int i=0;i<joinCount;i++)
161     {
162         string sid,ename;
163         cout<<endl<<"记录"<<i+1<<"-学号: ";cin>>sid;
164         cout<<"记录"<<i+1<<"-项目: ";cin>>ename;
165         m.joinEvent(sid,ename);
166     }
167     int querystu,querye;
168     cout<<endl<<"请输入要查询的人数: "; cin>>querystu;
169     while(querystu--)
170     {
171         string id;
172         cout<<endl<<"请输入要查询的学号: ";cin>>id;m.queryStudent(id);
173     }
174     cout<<endl<<"请输入要查询的项目数量: "; cin>>querye;
175     while(querye--)
176     {
177         string evename;
178         cout<<endl<<"请输入要查询的项目名
179 称: ";cin>>evename;m.queryEvent(evename);
180     }
    cout<<endl<<"总项目数: "<<SportsEvent::gets()<<endl;

```

```
181     m.show();  
182     return 0;  
183 }
```